



Programming an embedded system (Arduino) and
an interface for a high-throughput cell screening tool
- Final report

May 21, 2024

Table of contents

1	Introduction	3
1.1	Context	3
1.2	Company presentation	3
1.3	Problem to solve	3
2	State of the work	4
2.1	Technology used by MicroOmiX	4
2.2	Previous work	5
2.2.1	Choice of microcontroller	5
2.2.2	Human-machine interface	5
2.2.3	Sorting algorithm	6
3	Project study and progress	6
3.1	Client expectations	6
3.2	Functional analysis and specifications	6
3.3	Objectives and schedule for the year	6
4	Work carried out	9
4.1	Timer	9
4.2	ADC	10
4.3	DMA	11
4.4	Inter-core communication	12
4.5	M7 core - LabVIEW communication	13
4.6	Sorting algorithm	14
4.7	Final result	15

1 Introduction

The secretion of molecules by cells - such as proteins or antibodies - lies at the very heart of many biological processes: for example, the regulation of organ activity through different hormones, the establishment of appropriate immune responses, or even cell movement. The secretion mechanism has therefore been studied for several years, particularly with the aim of developing new therapies.

However, observing and analysing living systems and their behaviour at the cellular level requires cutting-edge technology. Recent years have seen the emergence of increasingly precise systems for carrying out these analyses, although they can still be lengthy and costly. One challenge in biotechnology is therefore to provide an effective, innovative and affordable approach in order to better understand the mechanisms of cellular secretion.

1.2 Company presentation

Founded in 2021, the start-up MicroOmiX develops a patented microfluidic technology originating from the University of Strasbourg and the CNRS, making it possible to develop and discover biotherapies. This would notably enable the rapid and effective development of new antibodies and improve cell therapy.

To achieve this, the company has developed a technology capable of individually and selectively analysing the role of a large number of cells in pathological or therapeutic mechanisms, while also recovering the cells and secreted molecules for therapeutic use, or more generally as a research tool.

Although the technology developed by MicroOmiX makes it physically possible to target cells using microfluidics, the data still needs to be exploited. The company decided to use an embedded system equipped with a microcontroller, subsequently connected to a computer in order to visualise the results.

The aim of this project is therefore to program this system so as to acquire the data produced by the tool and thereby sort the different types of cells. In particular, sorting must be fast - the required specifications will be detailed later in this document - while limiting errors. In addition, the start-up ultimately intends to commercialise its technology: a user-friendly graphical interface must therefore be developed to ensure optimal interaction between the user and the computer, in particular allowing the different parameters to be adjusted.

2 State of the work

Microfluidics, which manipulates fluids on the micrometre scale, makes it possible to rapidly sort a large number of cells according to their secretion profile. This technique can be used to study molecules secreted by cells - MicroOmiX has, for example, focused on antibody secretion. A single cell enclosed in a droplet can be kept alive for several days, giving it time to secrete biomarkers characteristic of antibodies.

The instrument used is capable of producing microdroplets containing a single cell together with a fluorophore system - whose location at the droplet surface differs depending on whether or not the cells secrete certain molecules - and of detecting this relocation with a photomultiplier as the droplet passes through a laser beam. The signal generated by fluorescence detection makes it possible to trigger a sorting event and recover populations of droplets containing cells that secrete a given type of antibody. The optical setup developed by the start-up is shown in Figure 1.

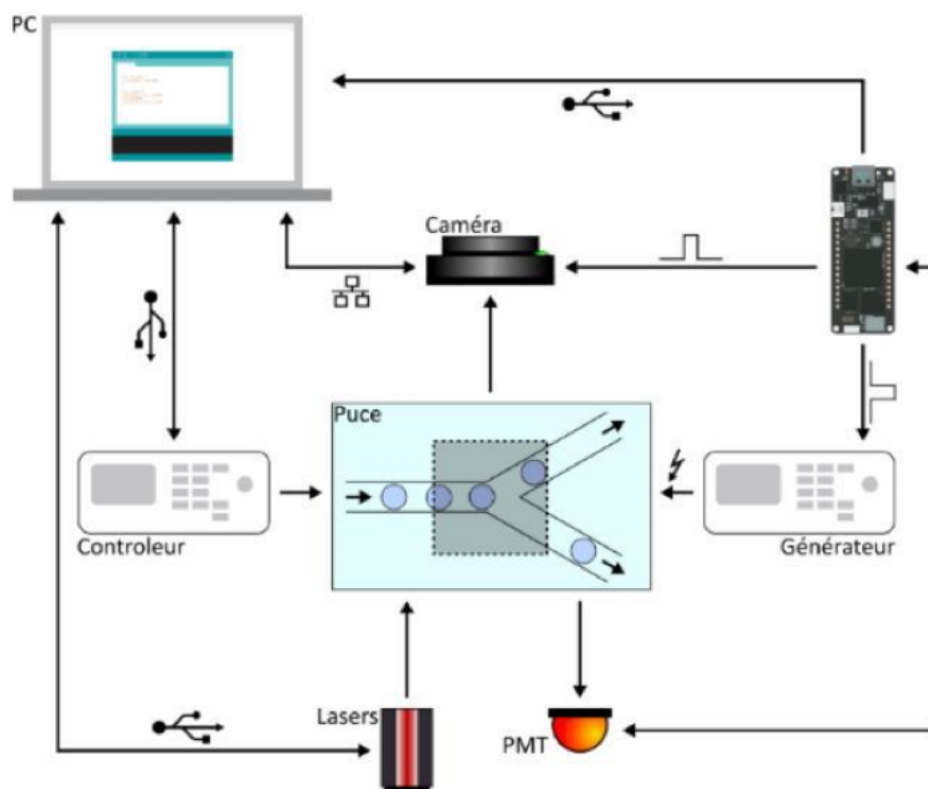


Figure 1 - Schematic diagram of the optical system used by MicroOmiX

For droplets containing an antibody-secreting cell, the associated signal has a so-called "rabbit-ear" shape. This is because the biomarkers are located only at the droplet interface, with the two peaks of the "rabbit ear" corresponding to the beginning and the end of the droplet's passage in front of the photomultiplier.

When the cell does not secrete antibodies, the fluorescence is distributed uniformly, producing a bell-shaped signal. The two types of signals are shown schematically in Figure 2.



Figure 2 - "Rabbit-ear" signal (left) and bell-shaped signal (right)

Our student group is not the first to work on this project: two other groups have worked on it in the past. In particular, a sorting algorithm based on an area-ratio calculation had already been written, although it could still be improved. This group also developed a Python interface and set up the processor architecture.

Subsequently, during his final-year project, an intern optimised the processor architecture and developed a LabVIEW interface. However, this interface is not fully functional and lacks clarity.

2.2.1 Choice of microcontroller

The board used for data management is an Arduino Portenta H7 equipped with a microcontroller capable of programming and controlling actuators from voltages supplied by sensors. It has 8 MB of RAM and 16 MB of external Flash memory. The board also includes three 16-bit analog-to-digital converters. It is built around a 32-bit dual-core processor: an ARM Cortex-M7 core clocked at 480 MHz and an ARM Cortex-M4 core clocked at 240 MHz. The two cores can communicate with each other and can be programmed independently. This board was chosen in particular for its ease of use - without having limited capabilities - and for its affordable price.

In order to manage the various parameters, such as the sorting frequency, while also observing the signal in real time, a graphical interface had to be developed. Although the first group worked on a Python interface, it quickly showed its limitations, particularly in terms of accessibility for the user. The company therefore recommended developing the interface using LabVIEW.

A LabVIEW interface was therefore developed by the previous intern, but because it was not fully functional, it needs to be modified.

2.2.3 Sorting algorithm

The droplet sorting algorithm is based on the SFR principle: Simple Fluorescence Ratio. It simply calculates the ratio of the areas between the first 20% of the signal generated by the droplet passage and the middle of the signal (40% to 60%). The method is therefore applied to the first part of the droplet because droplets are generally symmetrical, and it can be performed in real time, which is consistent with the objective of maintaining a high processing rate. However, this also means knowing the droplet size before it arrives.

After implementing the sorting algorithm, calibration must therefore be performed on a number of droplets defined by the user through LabVIEW in order to determine the average droplet size.

When we started the project, the system developed by MicroOmiX was capable of analysing between 300 and 400 droplets per second, with an acquisition frequency of 200 kHz. However, the start-up ultimately wanted to be able to analyse 1,000 droplets per second. One objective was therefore to improve and optimise the sorting algorithm. In addition, as mentioned above, the client intends to commercialise this technology. We therefore had to develop a human-machine interface that is easy to read and operate.

3.2 Functional analysis and specifications

The first part of the work consisted in establishing a functional analysis (Figure 3) and a set of specifications (Figure 4). These were drawn up after discussions with the client, who imposed a number of constraints, particularly regarding sorting and sampling frequencies.

3.3 Objectives and schedule for the year

The complexity of the subject required us to divide the work into separate parts in order to complete it. Based on our functional analysis and specifications, we identified four major areas that had to be addressed in the following order:

- Sample any analog signal at 100 kHz/16 bits, or 200 kHz/12 bits
- Temporarily write this signal to the Arduino board's shared memory

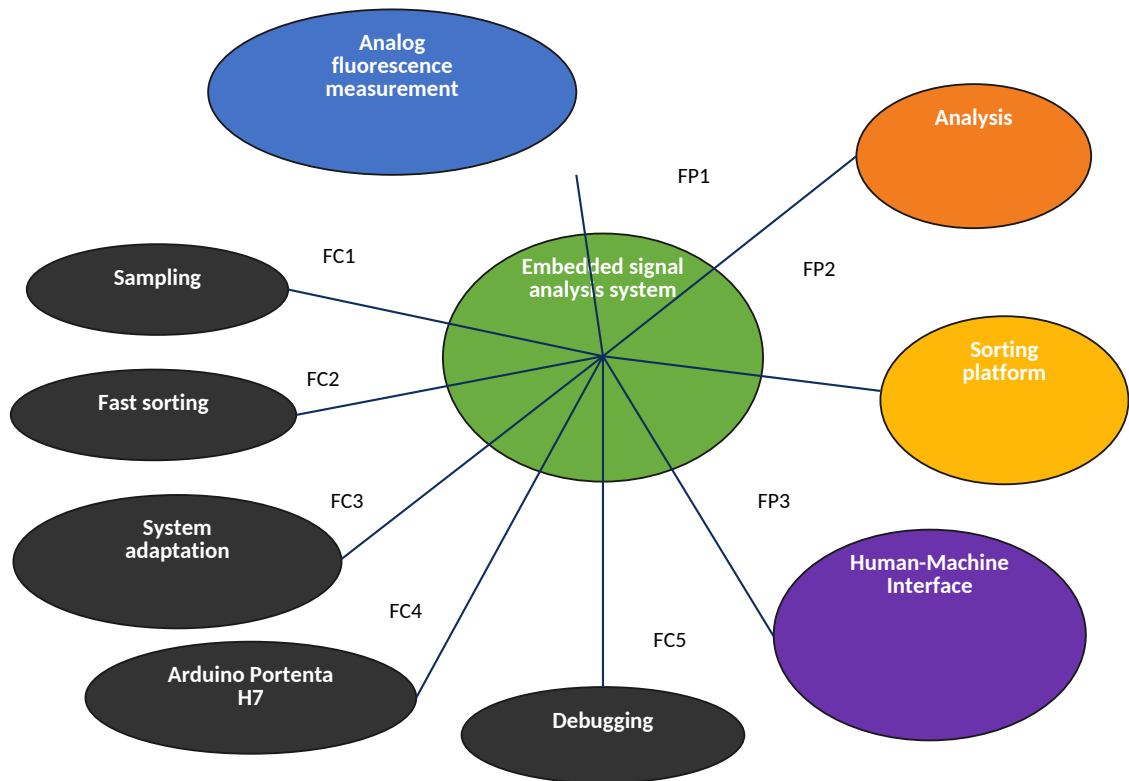


Figure 3 - Functional analysis performed after studying the project problem

Id	Function	Flexibility 0	Flexibility 1
FP1	Analyse	From the acquired signals, determine whether the signal is a "bell" or a "rabbit ear" using the SFR (Simple Fluorescence Ratio) criterion.	Use other analysis methods
FP2	Sort	Generate a 0/2 V analog square-wave signal at 15 kHz with an adjustable duty cycle.	
FP3	Control and display	Create a graphical interface on the computer allowing: - real-time signal display - adjustment of measurement-start thresholds - display of droplet-size and SFR histograms - identification of sorted droplets (camera capture trigger and signal display on the user interface)	
FC1	Sample	Dynamics: 16 bits Frequency: 100 kHz	Dynamics: 12 bits Frequency: 200 kHz
FC2	Sort quickly	5 ms between 2 cells (200 droplets/s)	1 ms between 2 cells (1,000 droplets/s)
FC3	Adapt to the signal	Variable cell width; requires an adapted threshold in order to correctly detect their size.	
FC4	Arduino Portenta H7	Recent board with little documentation. For high-frequency sampling, it is necessary to work directly with memory.	
FC5	Adopt a modular approach	Split the Arduino and LabVIEW code into blocks that can be validated independently.	

Figure 4 - Specifications obtained from the constraints imposed by the client

- Route it to the M4 core for analysis
- Route it to the M7 core and then to the computer in order to display and store it via LabVIEW

- Analysis:

- Numerical generation of model signals (rabbit-ear and dome)
- Sorting algorithm
- Display of the sorting signal
- Storage

- Sorting:

- Generation of the sorting signal
- Storage

Because task organisation is essential in this type of project, we established a schedule in the form of a Gantt chart (Figure 5) in order to meet the various deadlines as closely as possible.

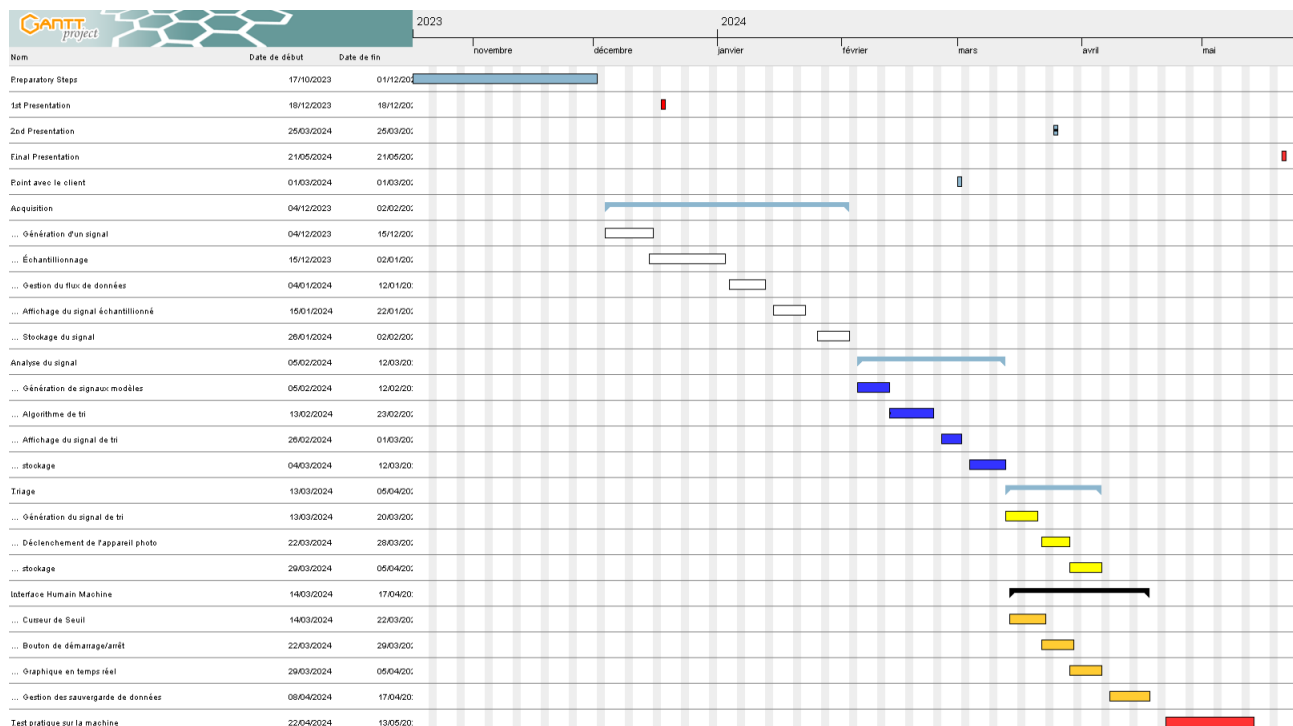


Figure 5 - Gantt chart

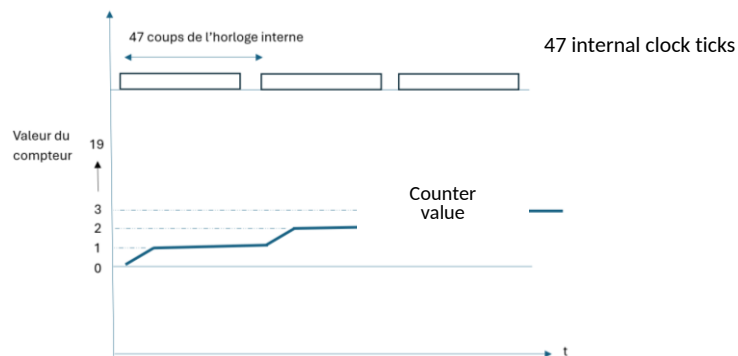
Timer:

From the internal clock signal of the M4 core, which runs at 240 MHz, we created a new timer1 clock signal running at 200 kHz. To do this, we used two timer registers:

Prescaler (or Clock Divider):

```
TIM1->PSC = 47;
```

- Every 47 ticks of the internal clock, a counter is incremented:



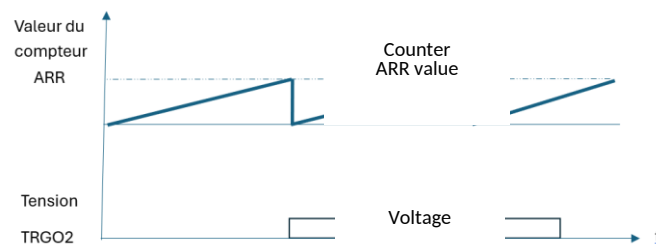
ARR (Auto Reload Register):

```
TIM1->ARR = 19;
```

- Resets the counter to 0 when the counter exceeds the ARR value 19.

```
SET_BIT(TIM1->CR2, TIM_CR2_MMS2_1);
```

- Configures an action each time the counter is reset.
Here, this action is to output a signal on the TRGO2 channel.



$$f_{timer1} = \frac{f_{coeur\ M4}}{PSC * (ARR + 1)} = \frac{240 * 10^6}{47 * 20} \approx 250\ KHz$$

ADC (Analog to Digital Converter):

Now that we have a signal at the correct frequency, we need to configure the ADC.

Single-Ended mode:

```
CLEAR_BIT(ADC1->DIFSEL, ADC_DIFSEL_DIFSEL_13);  
SET_BIT(ADC1->PCSEL, ADC_PCSEL_PCSEL_13);
```

- Disables differential mode on pin 13.

- Enables single-ended mode on pin 13: $U_{conv} = V_{pin13} - V_{ground}$

⚠ Processor pin 13 corresponds to Portenta pin PC3.

Discontinuous mode:

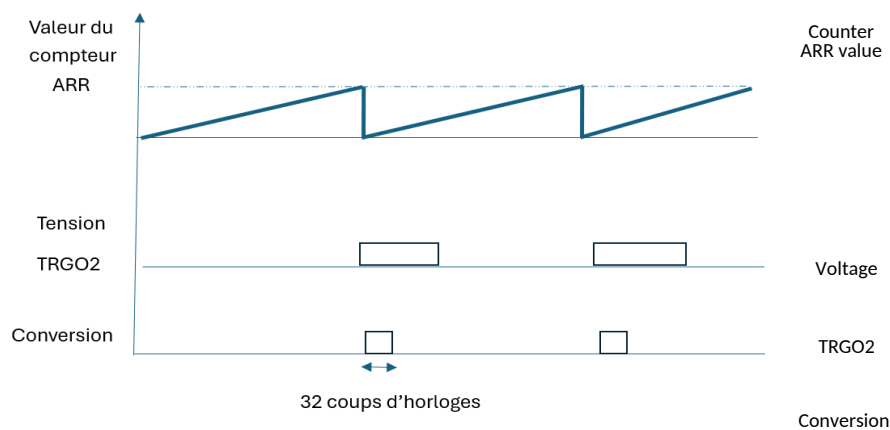
```
CLEAR_BIT(ADC1->CFGR, ADC_CFGR_CONT_Msk);
```

- Disables continuous mode.

In other words, the ADC operates in discontinuous mode: conversion occurs when a defined event is detected.

```
SET_BIT(ADC1->CFGR, ADC_CFGR_EXTEN_0 | ADC_CFGR_EXTEN_1 | ADC_CFGR_EXTSEL_1 |  
ADC_CFGR_EXTSEL_3);
```

Here, this corresponds to the rising edges of TRGO2 (our 200 kHz clock signal).



$$R_{conversion} = \frac{t_{conversion}}{T_{timer1}} = \frac{32}{20 \times 47} = 3\%$$

The conversion time represents only 3% of the Timer1 period. From a performance perspective, the sampling frequency could be increased considerably.

DMA (Direct Memory Access)

The ADC is now able to sample at the correct frequency (200 kHz). However, the data still has to be collected and redirected to the correct location.

Declaration of the source (the ADC):

```
DMA1_Stream1->PAR = (uint32_t)&ADC1->DR;
```

The ADC always returns the sampling result to the same address (&ADC1->DR). We must therefore ensure that the DMA does not increment the source address at the end of each transfer operation, hence:

```
CLEAR_BIT(DMA1_Stream1->CR, DMA_SxCR_PINCOS_Msk);
```

Declaration of the destination:

Double-buffer mode:

```
SET_BIT(DMA1_Stream1->CR, DMA_SxCR_DBM_Msk);
```

The destination will be 2 buffers (2 lists) of 20 16-bit elements defined in the shared structure discussed later. Its size is mutable and was chosen deliberately.

Declaration of these two buffers:

```
DMA1_Stream1->M0AR = (uint32_t)&buff->M7toM4_1;  
DMA1_Stream1->M1AR = (uint32_t)&buff->M7toM4_2;
```

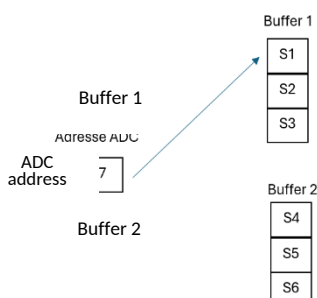
For both buffers to be filled, the destination memory address must be incremented.

```
SET_BIT(DMA1_Stream1->CR, DMA_SxCR_MINC_Msk);
```

Circular mode:

```
SET_BIT(DMA1_Stream1->CR, DMA_SxCR_CIRC_Msk);
```

This means that when both destination buffers are full, the destination address is reset to the first memory cell of the first buffer.

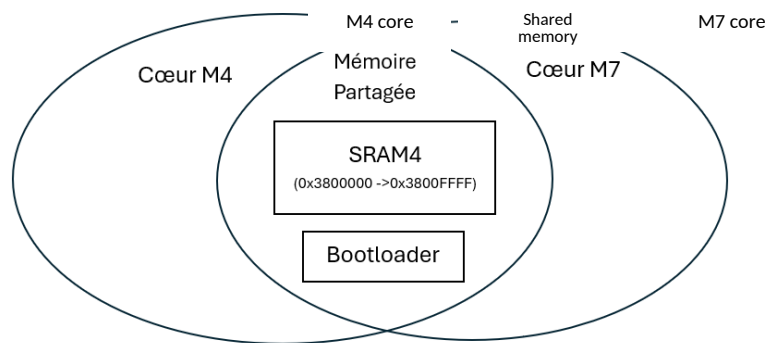


The main advantage of this storage method is that new samples can be accessed safely, without the risk of analysing them twice or having them disappear before there is time to process them. In practice, it is enough to retrieve the data from the buffer that is not currently the DMA target.

Inter-core communication:

Everything described above takes place only on the M4 core. However, in order to display the ADC data, it must be sent to the M7 core, which communicates with the PC (LabVIEW). To avoid introducing any delay, an ingenious method was found: shared memory. There is a memory area that can be read and modified by both cores. This area is not documented and was identified only after numerous tests.

SRAM4:



This usable shared-memory area (SRAM4) is limited in size. Care must therefore be taken not to exceed it, otherwise the bootloader could be damaged and the microcontroller rendered unusable.

Using shared memory:

It is sufficient to declare exactly the same data structure on both cores, specifying the same starting address located in shared memory (here 0x38001000).

Declaration of shared memory:

Cœur M4	Cœur M7
<pre>#ifdef CORE_CM4 struct shared_data { double sfr1; volatile long tfinal; int DropSizeSFR; • • }; volatile struct shared_data * const buff = (struct shared_data *)0x38001000; #endif</pre>	<pre>#ifdef CORE_CM7 struct shared_data { double sfr1; volatile long tfinal; int DropSizeSFR; • • }; volatile struct shared_data * const buff = (struct shared_data *)0x38001000; #endif</pre>

Using shared memory (M4 to M7):

Cœur M4	Cœur M7	Output Serial Monitor
<pre>#ifdef CORE_CM4 int taille_goutte = 150; buff->DropSizeSFR = taille_goutte; #endif</pre>	<pre>#ifdef CORE_CM7 int taille_goutte; taille_goutte = buff->DropSizeSFR; Serial.print(taille_goutte); #endif</pre>	Message (Enter to send message) 150

M7 Core - LabVIEW Communication:

The link between the M7 core and the PC uses the serial port (USB). The communication protocol is UART (Universal Asynchronous Receiver-Transmitter).

Data-flow problem:

The buffer data must be sent to LabVIEW before it is overwritten. As a reminder, this corresponds to 20 16-bit integers.

Buffer fill time:

For this, we have the time required to fill the other buffer:

$$T_{remplissage} = 20 * T_{ech} = \frac{20}{250 \text{ kHz}} = 80 \mu\text{s}$$

Determining the appropriate baud rate:

The hardware constraints impose communication in the following format: 8 data bits, 1 start bit and 1 stop bit.

To optimise transmission time, the 16-bit integers are split into 2 bytes: one high-order byte and one low-order byte.

```
val = 1234; //0000010011010010 (binary)
buffer[0] = val & 0xFF; //Low-order byte (11010010)
buffer[1] = (val >> 8) & 0xFF; //High-order byte (00000100)
Serial.write(buffer, 2);
```

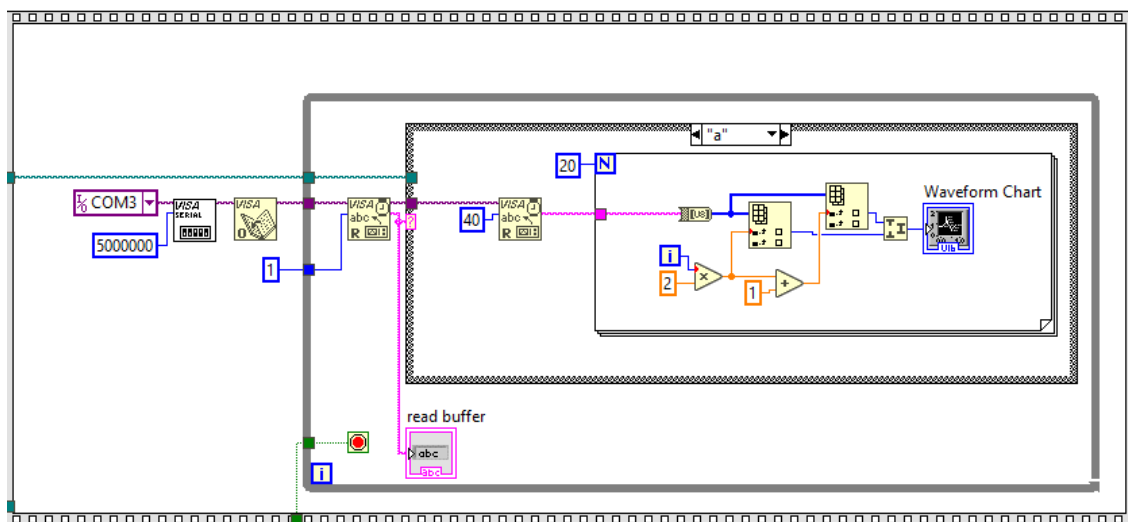
LabVIEW must also be informed of the type of data it is about to read by sending an ASCII character (8 bits) as a header.

$$T_{empty} = [1 \text{ (start bit)} + 20 \text{ (list size)} \times 2 \times 8 \text{ (16 bits in 2 bytes)} + 1 \text{ (stop bit)} + (1 + 8 + 1) \text{ (header)}] / \text{BaudRate (bit rate)}$$

We can estimate that $T_{empty} = 90\% \times T_{fill}$ is a sufficient safety margin to avoid any risk of data loss.

$$\text{BaudRate} = (1 + 20 \times 2 \times 8 + 1 + 10) / T_{empty} = 4.7 \text{ MBaud} \approx 5 \text{ MBaud}$$

Retrieval in LabVIEW:

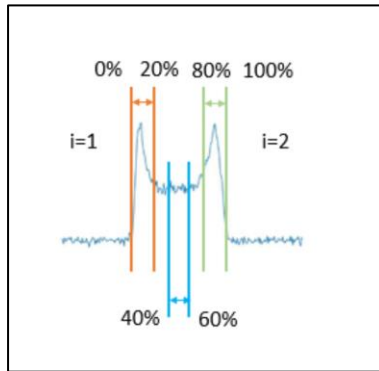


Sorting algorithm:

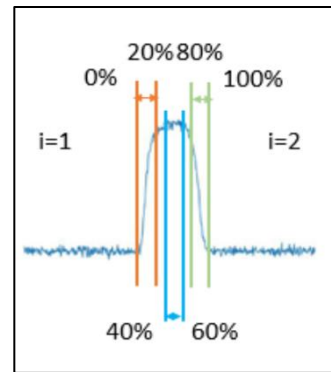
SFR (Simple Fluorescence Ratio):

It is simply the ratio between two areas: the first fifth of the curve and the third fifth. It makes it possible to distinguish rabbit-ear signals from dome signals.

- SFR > 1: this is a rabbit-ear signal.
- SFR < 1: this is a dome signal.



Rabbit-ear signal

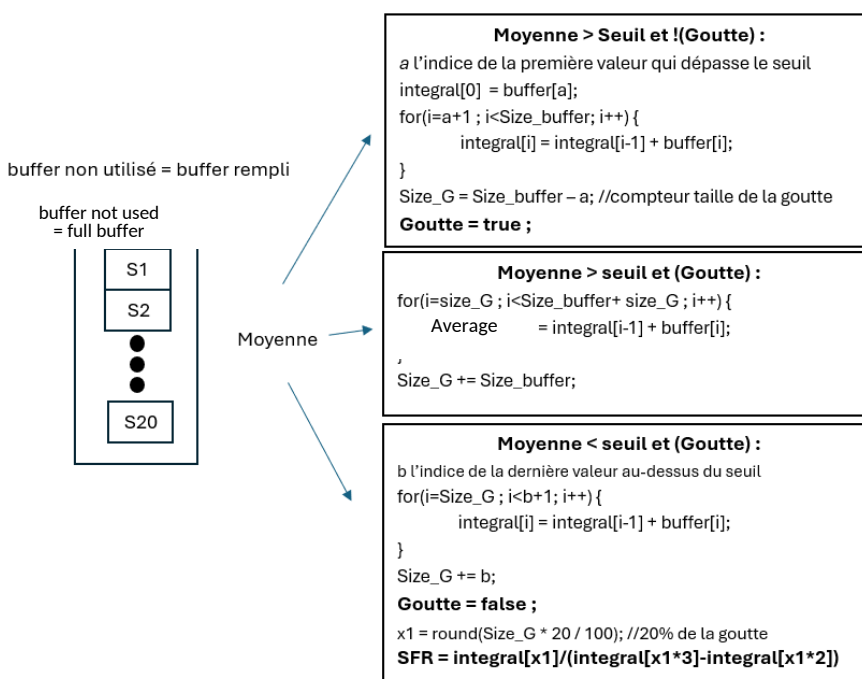


Dome signal

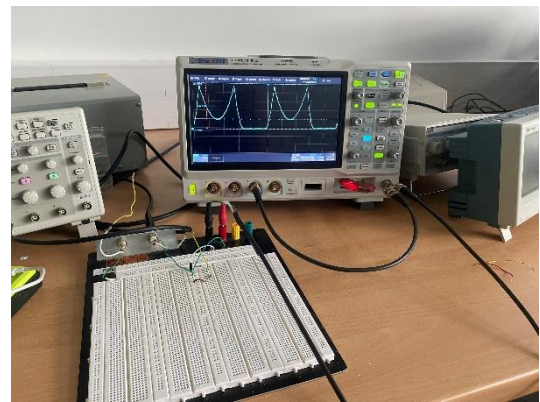
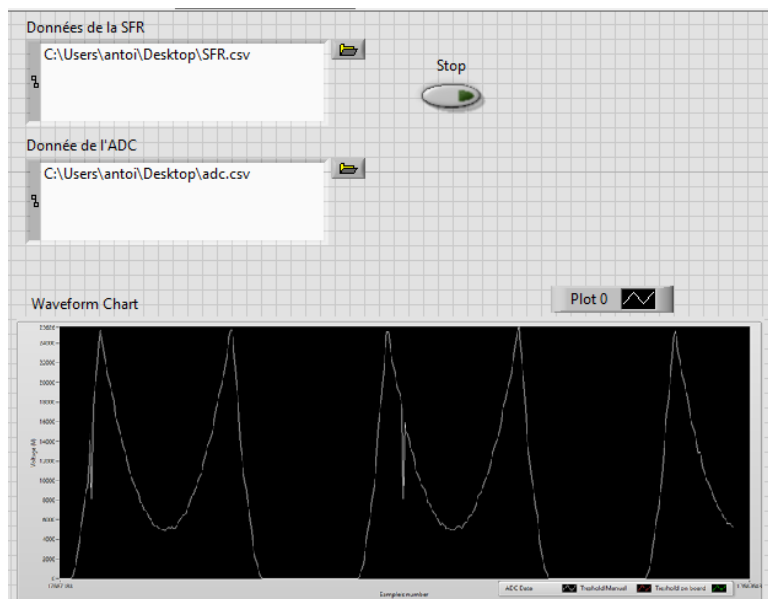
$$SFR_1 = \frac{\int_0^{t_g*0,20} fluo(t)dt}{\int_{t_g*0,40}^{t_g*0,60} fluo(t)dt} \text{ with } t_g: \text{droplet size}$$

Algorithm:

Must be able to provide 3 values: the time at which the droplet is detected, its size and its SFR.



Final result:



	A	B	C	L
1	Temps	Taille	SFR1	
2	33184	148	2	
3	33188	155	2	
4	33191	140	2	
5	33196	148	2	
6	33198	151	2	
7	33200	155	2	
8	33201	147	2	
9	33203	150	2	
10	33206	139	2	
11	33210	140	2	
12	33213	146	2	

Summary:

Acquisition:

- The required 200 kHz/16-bit sampling rate is successfully achieved.
- Display stability is good, although there are a few fluctuations; the baud rate may need to be increased slightly.

Sorting:

- The various tests performed on slightly noisy model signals with the sorting algorithm give 100% accuracy for the SFR. However, these are very clean signals. It would have been necessary to test the program with real analog signals.

Storage:

- Data storage works completely: all data reaching LabVIEW are written to .csv files.

Sorting signal:

- Due to a lack of time, and in agreement with the client, we decided to focus more on data acquisition and analysis. It was therefore not implemented.

Possible improvements:

It is clear that the board is not being pushed to its full potential. The ADC rate could be increased very significantly, but the communication protocol would probably need to be changed, moving to Ethernet connectivity with much higher baud rates.

Regarding the sorting algorithm, more sophisticated methods could be used, such as derivatives or the Fourier transform.